

Subproject 1: Classification of ID-Cards



Image and Video Analysis

G.E.E.T. 4th year

José Luis Alba Castro

Description

- **Goal:** Code a solution that recognizes different types of ID-Cards in any orientation and outputs the name of the type stored in the system
- **Images for developing:** You should test at least 5 different types of ID-Cards captured with your mobile phone or webcam. Make sure to take several pictures of each type with different environment conditions.
- **Restrictions on the Input of the solution:**
 - **Easy static scenario:** ID-Cards captured on a flat uniform surface. Two options: the input is a picture of the ID-Card, the input is a video of an ID-Card left in a flat surface
 - **Difficult static scenario:** the same with any background (other restrictions can be applied)
 - **Hand-held scenario:** the ID-Card is shown to a webcam, with uniform or complex background (other restrictions can be applied)
- **Operation mode:**
 - Initial state: no ID-Cards are stored in the system
 - Any new image entered (disk or webcam) in the program, is compared to the existing ones. If there is a match, the name of the type is produced. If not, the program prompts the user to introduce the name of the ID-Card type in the system.

- **Tips:**
 - Try different Color Spaces to represent the ID-Cards.
 - Try different histogram distances, number of bins, range of histogram and number of channels.
 - Make the histogram invariant to scale.
 - Study the influence of the background to eliminate it (static)
 - How far can you go without segmenting the ID-Card?
 - Try to segment the ID-Card to account only for its colors.
 - Hough transform (theory) → [HoughLines](#), [HoughLinesP](#)
 - Closed Contours → [findContours](#), [approxPolyDP](#)
 - How far can you go using only histograms as image descriptors?
 - Start exploring other options like CNN models, but this will be explained later on...
- **General working rules:**
 - You can use OpenCV examples or internet code and modify them if allowed. You can ask to AI tools. You can share code with your classmates through Moovi.
 - You should indicate the origin of all your code in the final report, otherwise you can be penalized.

- **Scoring rules:**

Independently on the version you deliver, your subproject should fulfill the next requirements:

- 1) The programs should run in Windows11. Make sure to give the correct instructions for me to run it from the source files as if I were a client. Provide a “requirements.txt” file (for pip) or a “environment.yml” file (for conda) to indicate the python packages that are needed, so I can create an environment in python to run your solution. I advice you to simulate a second installation in a friend's computer. If you are using a library/package/dataset that is not easy to find in standard repositories, provide it.
- 2) The solution should run as explained in the Description slide. Any modification of the rules should be explained and justified. The PowerPoint should include the next 4 points:
 - you should specify how the program(s) can be used by a third party,
 - you should explain what the code does, part by part, and highlight the part of the code that has been reused indicating where you got it from,
 - you should indicate under what conditions the method works and it doesn't work,
 - you should include images to explain how the methods work and some examples where it doesn't work and why.
- 3) The classifier should work well (and not halting!) for the Selected ID-Cards. That means that the program always gives an answer independently on the number of ID-Card types (≥ 5). Specifically check that:
 - a) the program always asks the type when the ID-Card is not in the system
 - b) the program returns the ID-Card type if it is already in the system
 - c) the program outputs a message when it doesn't find a rectangular object in the image.

The maximum score for the Easy static scenario is 7 (image) or 8 (video)

The maximum score for the Difficult static scenario is 9 (video)

The maximum score for the Hand-held scenario is 10 (video)

- **Challenge:**

Independently on the version you deliver (easy/difcult/hand-held) you can work for this challenge by building a Python function that receives an input **image** and process it to find a quasi-rectangular shape corresponding to the object of interest in the image. The best function in terms of robustness in different scenarios and processing speed will gain 1 extra point to share among the authors in the proportion they agree.

Delivery rules

* Python function:

- input image of any size and file type
- if the image contains a quasi-rectangular object, return a MASK image.
The pixel values of the mask should be 0 outside the rectangle and 1 inside it.
- if the image doesn't contain a rectangular object the mask should be 0.

* A PowerPoint explaining the method:

- you should specify how the function/class can be used by a third party
- you should highlight the part of the code that has been copied, the source where you get it from and the main ideas behind the method, so you show that you understood it.
- you should indicate under what conditions the method works and in what cases it doesn't work, with some illustrative examples.

- The winning group will have to make an oral presentation explaining the method and using the delivered PowerPoint.
- The code can be used for the other groups to improve their Project